American Tiger LLC
5900 Balcones Drive Suite 100
Austin, TX 78731

TAKE HOME ASSIGNMENT – BACK END DEVELOPER

ASSESSMENT & EVALUATION FOR FULL-TIME CANDIDATES - June 2026

Project Overview: Operations API, RAG and chatbot implementation.

Primary Objective: To consider your candidature for the Python Backend developer position.

This Take-home Project will give your potential Team Member & other senior stakeholders an opportunity to assess & evaluate your technical skills and decide for potential next steps, if any.

Technical Objective: This project involves building the backend infrastructure for a Chatbot. Candidates will create a robust set of REST APIs that allow users to register, upload private documents to build a personal knowledge base, and query those documents using an AI assistant.

Project Start Date: Wednesday, June 17th, 2026

Last Project Submission Date: Saturday, June 20th, 2026

Technical Overview

1. LangChain (RAG Framework & Vector Workflows)

LangChain is an open-source orchestration framework designed to simplify the construction of applications powered by Large Language Models (LLMs).

Role in this Project: Instead of writing custom semantic search pipelines, you will leverage LangChain's built-in modules to handle the core mechanics of Retrieval-Augmented Generation (RAG). This includes using text splitters (e.g., RecursiveCharacterTextSplitter) to chunk uploaded files, creating vector store abstractions (e.g., Chroma, FAISS, or Pinecone) to index data, and orchestrating the final RAG chain to inject retrieved context directly into the LLM prompt.

2. OpenRouter (Unified LLM API & Free-Tier Access)

OpenRouter is an API aggregator that provides a single, unified interface to interact with dozens of different open-source and commercial AI models (including models from OpenAI, Anthropic, Meta, and Google).

Role in this Project: To ensure you can complete this take-home project completely free of charge, you will use OpenRouter as your LLM gateway. OpenRouter offers a robust selection of free-tier models (such as Mistral 7B, Gemma, or OpenChat variants). You will configure your LangChain LLM class to route requests through OpenRouter's API endpoint, removing the need for paid personal API keys.

Project Requirements:

Part 1: Authentication & Subscription Management

All endpoints under this section handle user access control.

APIs to be Created

User Registration

- **Endpoint:** /api/register/
- **Method:** POST
- **Request Format:** Multipart Form-Data or JSON Body
- **Example Request:**
- JSON

```
{
 "email": "example@gmail.com",
 "password": "password123"
}
```

-
- **Response:**
  - **On Success (201 Created):**
  - JSON

```
{
 "message": "Registration successful",
 "user_id": "<user_id>"
}
```

  - **On Failure (400 Bad Request):**
  - JSON

```
{
 "error": "User with this email already exists."
}
```

  -

User Login

- **Endpoint:** /api/login/
- **Method:** POST
- **Request Format:** Multipart Form-Data or JSON Body
- **Example Request:**
- JSON

{

"email": "example@gmail.com",

"password": "password123"

}

-
- **Response:**
    - **On Success (200 OK):** Returns an access token to be used in the Authorization header.
    - JSON

{

"message": "Login successful",

"token": "<jwt_access_token>"

}

    -
    - **On Failure (401 Unauthorized):**
    - JSON

{

"error": "Invalid email or password"

}

    -

JWT Token Middleware

Ensure all protected routes below require a valid JWT token passed via the Authorization header:

Authorization: Bearer <jwt_access_token>

Part 2: Document Management & Vector Storage (LangChain)

Endpoints under this section allow users to build their private knowledge base. Uploaded files must be processed, chunked, embedded, and indexed inside a vector store via LangChain. This process should be handled by a celery worker

APIs to be Created

Document Upload API

- **Endpoint:** /api/documents/upload/
- **Method:** POST
- **Headers:** Authorization: Bearer <jwt_access_token>

- **Request Format:** Form-data with a file field.
- **Example Request:**
- Plaintext

  Form-Data:

  file: @knowledge_base.pdf

-
- **Response:**
  - **On Success (202 Accepted):** Task is accepted for asynchronous ingestion.
  - JSON

```
{
 "message": "Document uploaded and ingestion started",
 "document_id": "<document_id>",
 "task_id": "<task_id>"
}
```

  -
  - **On Failure (400 Bad Request):**
  - JSON

```
{
 "error": "Invalid file format. Only PDF, TXT, and Markdown files are allowed."
}
```

  -

Document Ingestion Status API

- **Endpoint:** /api/documents/status/
- **Method:** GET
- **Headers:** Authorization: Bearer <jwt_access_token>
- **Request Parameters:** task_id=<task_id>
- **Response:**
  - **Task Running:**
  - JSON

```
{
 "task_id": "<task_id>",
 "status": "PROCESSING"
```

}

- **Task Success:**
- JSON

{

"task_id": "<task_id>",

"status": "SUCCESS",

"message": "Document successfully parsed, embedded, and indexed in vector storage."

}

- **Task Failed:**
- JSON

{

"task_id": "<task_id>",

"status": "FAILURE",

"error": "Failed to parse document content."

}

Expected Background Pipeline Operations

- **Text Extraction:** Read raw content from the uploaded file type (PDF, TXT, or MD).
- **Chunking:** Use LangChain's text splitters (e.g., RecursiveCharacterTextSplitter) to split document content into readable, contextual chunks.
- **Vector Ingestion:** Calculate embeddings via an embedding model open-source or commercial, and store the vector chunks inside an isolated namespace linked to the uploading user's user_id.

Part 3: RAG Chat Engine & Credit Consumption

This section connects the retrieved context to an LLM, and generate an answer.

APIs to be Created

RAG Query API

- **Endpoint:** /api/chat/query/
- **Method:** POST
- **Headers:** Authorization: Bearer <jwt_access_token>
- **Request Format:** JSON Body

- **Example Request:**
- JSON

```
{
  "query": "What is the cancellation policy mentioned in the employee handbook?"
}
```

-
- **Processing Rules:**
  - Use LangChain's Vector Store Retriever to search for context matches inside document collections belonging to the user.
  - Run the combined query and context prompt through the LLM.
- **Response:**
  - **On Success (200 OK):**
  - JSON

```
{
  "answer": "The cancellation policy requires a written notice 14 days prior...",
  "tokens_consumed": 142,
}
```

Bonus Features (for bonus points)

- **Chat Continuation API:** Implement an endpoint that allows users to continue an existing conversation by providing a chat_id or previous message history.
- **Server-Sent Events (SSE) Streaming:** Use Server-Sent Events to stream responses in real time, providing a smoother and more responsive chat experience.

Part 4

Dockerizing and Finalization

- Use Docker and Docker Compose to package your web application, database, Redis, Celery and dashboard services.

- When showcasing your application, include the necessary Docker commands to run the system in a README file.

- Document your API using Postman, Bruno, OpenAPI, or another widely used API documentation tool.

- Ensure the README also includes clear instructions on how to set up and run the entire application.

Please submit your Final Submission / Paper to:

Mr. Rajesh Behera, Front-End Developer at: rajesh.behera@ravid.cloud

Closing Notes: Should you have any **technical questions** at any time, please contact:

Mr. Phuong Tung Tran, Back-End Developer & Full Stack Development Team at phuongtung.tran@ravid.cloud / OR **Mr. Truong Vinh Phuoc "Matthew"**, Back-End Developer & Full Stack Development Team at phuoc.truong@ravid.cloud